

MSDS 535

Week 6 – Chapter 6

Overview and Objectives

- Understand Classification: Basic concepts and methods.
- Analyze Decision tree induction.
- Understand Bayes classification methods.
- Discuss Lazy learners (or learning from your neighbors).
- Interpret Linear classifiers.
- Understand Model evaluation and selection.
- Develop Techniques to improve classification accuracy.
- Summarize classification basic concepts and methods.

Basic Concepts

- Classification is a form of data analysis that extracts models to predict categorical class labels.
- Examples of classification applications include bank loan categorization, fraud detection, sentiment analysis, and more.
- Classification belongs to supervised learning and is related to other data mining tasks.
- There are two major types of prediction problems: classification and numeric prediction (regression).
- Classification and regression are closely related, and some classification techniques can be modified for regression.

What is Classification?

- Classification involves constructing models or classifiers to predict categorical class labels.
- Class labels are discrete, unordered values representing categories.

General Approach to Classification

- Classification is a two-step process: learning and classification.
- In the learning step, a classifier is built using a training set with known class labels.
- Supervised learning involves knowing the class labels during training, unlike unsupervised learning.

General Approach to Classification

- Various forms of supervised learning, including semi-supervised and zero-shot learning, are introduced.
- The learning step involves building a mapping or function to predict class labels.
- In the classification step, the model's accuracy is evaluated using a test set, and if acceptable, it's used for classifying new data.

Decision Tree Induction

- Decision tree induction involves learning decision trees from class-labeled training data.
- Decision trees are tree structures with internal nodes representing attribute tests and leaf nodes holding class labels.
- Decision trees are used for classification and can be converted into classification rules.
- Decision tree classifiers are popular due to their simplicity, speed, and good accuracy.
- The construction of decision trees does not require domain knowledge.

Decision Tree Induction

- Decision trees can handle multidimensional data.
- Decision tree algorithms adopt a top-down, recursive approach.
- The algorithm involves selecting the best attribute for splitting data based on attribute selection measures.
- Decision trees can be binary or non-binary, depending on the attribute selection measure.
- Tree pruning is used to improve classification accuracy by removing noisy branches.

Decision Tree Induction

- Decision tree induction creates decision trees from class-labeled training data.
- Key algorithms include ID3, C4.5, and CART.
- The process is top-down and recursive.
- Its goal is to divide the attribute space effectively to group similar data points.
- It starts with a single node representing the whole training set.

Decision Tree Induction

- Recursive partitioning continues until termination conditions are met:
 - All tuples in a partition belong to one class.
 - No attributes are left for further partitioning.
 - A branch becomes empty due to data absence.
- Computational complexity: $O(n \times |D| \times \log(|D|))$.

Decision Tree Induction

- Decision trees serve as predictive models for classifying new data.
- Used in medicine, finance, astronomy, and more due to simplicity and accuracy.
- Incremental versions update trees with new data.
- Algorithm differences include attribute selection and pruning strategies.

Attribute Selection Measures

- Attribute selection measures help choose the best splitting criterion for data partition D in class-labeled training tuples.
- Ideally, the chosen criterion leads to pure partitions (all tuples in a partition belong to the same class).
- Attribute selection measures are also known as splitting rules as they guide how tuples at a node are split.
- They rank attributes based on their relevance to training tuples.

Information Gain

- ID3 uses information gain for attribute selection.
- It minimizes information needed for classifying tuples in resulting partitions.
- Information gain is based on the concept of entropy, measuring information content.
- Information needed to classify a tuple in D is given by $\text{Info}(D)$, also known as entropy.
- Partitioning on attribute A with v distinct values results in subsets $\{D_1, D_2, \dots, D_v\}$.

Information Gain

- $\text{InfoA}(\mathbf{D})$ measures information required for classification after partitioning.
- Information gain is $\text{Gain}(A) = \text{Info}(\mathbf{D}) - \text{InfoA}(\mathbf{D})$.
- Attribute A with the highest information gain is selected as the splitting attribute.
- For continuous-valued attributes, split-points are considered, and the one with the minimum expected information requirement is chosen.

Gain Ratio

- C4.5 uses gain ratio to counter information gain's bias towards multivalued attributes.
- Gain ratio normalizes information gain using "split information."
- Split information, $\text{SplitInfoA}(\mathbf{D})$, quantifies the potential information generated by splitting data into v partitions.
- $\text{GainRatio}(A) = \text{Gain}(A) / \text{SplitInfoA}(\mathbf{D})$.
- The attribute with the maximum gain ratio is chosen as the splitting attribute.

Gini Impurity

- Gini impurity is used in CART.
- It measures impurity based on class probabilities in partition D .
- $\text{Gini}_A(D)$ calculates impurity for binary splits on attribute A .

Gini Impurity

- The attribute with the minimum Gini impurity is selected as the splitting attribute.
- For continuous-valued attributes, split-points are considered, and the one with the minimum Gini impurity is chosen.
- Gini impurity is computationally efficient compared to information gain.

Other Attribute Selection Measures

- CHAID uses a χ^2 test for independence as an attribute selection measure.
- C-SEP and G-statistic are alternative measures.
- MDL-based measures favor simplicity and are unbiased toward multivalued attributes.
- Some measures consider multivariate splits, creating new attributes based on combinations of existing ones.

Other Attribute Selection Measures

- The choice of the best attribute selection measure depends on various factors, including the desired tree depth and tree complexity.
- No single measure is significantly superior, and most provide satisfactory results in practice.
- This section discusses popular attribute selection measures but does not cover all possible measures.
- Different measures may be preferred depending on specific use cases and data characteristics.

Tree Pruning

- Decision trees may contain branches reflecting noise or outliers in training data.
- Tree pruning methods aim to address overfitting by removing unreliable branches.
- Pruned trees are typically smaller, simpler, easier to understand, and perform better on independent test data.

Prepruning

- Prepruning involves halting tree construction early at a node, making it a leaf.
- Measures like statistical significance, information gain, or Gini impurity are used to assess split quality.
- If the split quality falls below a specified threshold, partitioning is halted.

Postpruning

- Postpruning is more common and involves removing subtrees from a fully grown tree.
- A subtree is replaced by a leaf with the most frequent class label within that subtree.
- The cost complexity pruning algorithm in CART is an example of postpruning.

Pessimistic Pruning

- Pessimistic pruning, used in C4.5, estimates error rates without the need for a pruning set.
- It adjusts error rates from the training set to counter the inherent optimism in training-based estimates.

MDL-Based Pruning

- MDL-based pruning minimizes the number of bits required to encode a tree, following the MDL principle.
- It prefers simpler tree representations.

Pruning Continued

- Some approaches combine prepruning and postpruning techniques for a balanced trade-off between efficiency and reliability.
- No single pruning method is universally superior; the choice depends on factors like tree depth and data characteristics.

Challenges in Decision Trees

- Pruned trees can still be large and complex, leading to repetition and replication issues.
- Repetition occurs when an attribute is repeatedly tested along a branch.
- Replication happens when duplicate subtrees exist in the tree.
- These problems can hinder accuracy and comprehensibility.

Alternative Approaches

- To address complexity issues, multivariate splits, which consider attribute combinations, can be used.
- Another option is to represent knowledge in rule form rather than decision trees, offering more concise and interpretable models.

Overview of Bayesian Classifiers

- Bayesian classifiers are statistical classifiers that predict class membership probabilities for given data.
- They rely on Bayes' theorem for classification.
- Naïve Bayesian classifiers assume class-conditional independence among attributes.

Bayes' Theorem

- Bayes' theorem calculates the posterior probability $P(H | X)$ of a hypothesis H given evidence X .
- $P(H | X)$ is the probability that a hypothesis holds given the observed data X .
- $P(H)$ is the prior probability of H .
- $P(X | H)$ is the conditional probability of X given H (often referred to as likelihood).
- $P(X)$ is the prior probability of X .

Bayesian Classifier in Practice

- Given m classes, the Bayesian classifier calculates posterior probabilities $P(C_i | X)$ for each class.
- It predicts that the tuple X belongs to the class with the highest posterior probability.

Optimality and Risk

- In theory, the Bayesian classifier is optimal, having the lowest classification error rate compared to other classifiers.
- It can still make incorrect predictions, but it minimizes the overall risk (probability of incorrect prediction) compared to other classifiers.

Challenges in Bayesian Classifiers

- Estimating conditional probabilities $P(X | C_i)$ is challenging, especially when there are many attributes.
- The attribute value space can be exponential, making parameter estimation difficult.
- Naïve Bayesian classifiers address this by assuming independence among attributes.

Why Not Use Bayes Classifier Directly?

- Bayes' theorem requires estimating conditional probabilities, which can be computationally intensive.
- Estimating priors is straightforward, but estimating conditional probabilities can be challenging.
- Naïve Bayesian classifiers provide a simpler solution by making the independence assumption among attributes.

Naïve Bayesian Classification

- Naïve Bayesian classifier follows the same procedure as the Bayes classifier but estimates conditional probabilities differently.
- Data Representation
 - Training set D consists of tuples with associated class labels.
 - Each tuple is represented by an n -dimensional attribute vector, $X = (x_1, x_2, \dots, x_n)$.
 - There are m classes, $C_1, C_2, \dots, C_m$.

Naïve Bayesian Classification

- Classification Process
 - Predicts that X belongs to the class C_i with the highest posterior probability.
 - Uses Bayes' theorem: $P(C_i | X) = P(X | C_i) * P(C_i) / P(X)$.
- Class Prior Probabilities
 - If class priors are unknown, assume classes are equally likely.
 - $P(C_i)$ may be estimated by $|C_i, D| / |D|$.

Naïve Bayesian Classification

- Class-Conditional Independence Assumption
 - Assumes conditional independence among attributes given the class label.
 - Simplifies the estimation of $P(X | C_i)$.
- Estimating Conditional Probabilities
 - For categorical attributes: $P(x_k | C_i) = (\text{Count of tuples with } x_k \text{ for } A_k \text{ in class } C_i) / (\text{Total count of tuples in class } C_i)$.
 - For continuous-valued attributes: Gaussian distribution is assumed for each class C_i .
- Prediction of Class Label
 - Compute $P(X | C_i) * P(C_i)$ for each class C_i .
 - Predict the class C_i with the maximum value of $P(X | C_i) * P(C_i)$.

Effectiveness of Naïve Bayesian Classifier

- If the class-conditional independence assumption holds, it can be optimal with minimal classification error.
- In practice, its accuracy depends on the validity of assumptions and available probability data.
- Comparable to decision trees and selected neural network classifiers in some domains.

Naïve Bayesian Classification

- Handling Probability Values of Zero
 - Use Laplacian correction (add-one smoothing) to avoid zero probability values.
 - Assume a large training database to minimize the impact of added counts.
- Limitations and Sophistication
 - Class-conditional independence assumption is a limitation.
 - More sophisticated methods like Bayesian networks can address this limitation.

Lazy Learners (Learning from Neighbors)

- Lazy learners wait until the last minute to construct a classification model.
- Store training data and perform classification based on similarity to stored data.
- Suitable for incremental learning and complex decision spaces.

k-Nearest-Neighbor Classifiers

- Based on the idea of learning by analogy.
- Training tuples are points in an n -dimensional attribute space.
- Classify an unknown tuple by finding its k -nearest neighbors in attribute space.

Distance Metric

- Euclidean distance commonly used for numeric attributes.
- For nominal attributes, compare values (0 for identical, 1 for different).
- Missing values may be treated as maximum possible difference.

Choosing the Value of k

- Determine experimentally based on a test set.
- Larger k values for larger training datasets.
- As k approaches infinity, error rate approaches Bayes error rate.

Speeding up k-Nearest-Neighbor Classifiers

- Parallelization, presorting, and search trees reduce computational cost.
- Partial distance calculations and editing can save time.
- Locality-sensitive hashing (LSH) is another technique for efficient search.

Challenges and Modifications

- Equal weight to all attributes can lead to poor accuracy with noisy or irrelevant attributes.
- Attribute weighting and pruning noisy data are possible modifications.
- Choice of distance metric and efficiency trade-offs are challenges.

Case-Based Reasoning (CBR)

- Uses a database of problem solutions (cases) to solve new problems.
- Searches for similar cases and combines their solutions for new cases.
- Challenges include finding a good similarity metric and combining solutions.

Efficiency and Case Editing

- Efficiency vs. accuracy trade-off with increasing stored cases.
- Editing the training database to remove redundant or unhelpful cases can improve performance.

Linear Classifiers

- Various classifiers generate complex decision boundaries (e.g., decision tree, k-nearest-neighbor).
- Linear decision boundaries offer advantages like efficient computation, better generalization, and interpretability.

Linear Regression

- **Linear Mapping:** Linear regression predicts a continuous value based on independent attributes, assuming a linear combination of inputs.
- **Weight Vector:** It involves determining a weight vector (w) and bias scalar (b) that minimizes the least square loss function.
- **Single vs. Multiple Attributes:** For single attribute, w and b can be computed directly; for multiple attributes, introduce a dummy attribute and compute w using matrix operations.
- **Loss Minimization:** The goal is to minimize the sum of squared differences between predicted and observed values, leading to the optimal weight vector w and bias scalar b .

Turning Linear Regression to Classification

- Binary Classification: Perceptron modifies linear regression for binary classification tasks, where $y = 1$ for positive tuples and $y = 0$ for negative ones.
- Prediction: Predicts the class label ($\hat{y}$) based on the sign of the output of the linear regression model.
- Training Algorithm: Utilizes an iterative algorithm that updates the weight vector (w) for misclassified tuples.
- Linear Separability: Guarantees finding a hyperplane decision boundary for linearly separable data; fails to converge for non-separable data.

Logistic Regression

- Logistic regression predicts binary class labels and provides prediction confidence.
- It utilizes the sigmoid function to transform real numbers into values between 0 and 1.
- Logistic regression calculates posterior probabilities using sigmoid: $P(\hat{y}^i = 1 \mid \mathbf{x}_i, \mathbf{w}) = \sigma(\mathbf{w}^T \mathbf{x}_i)$.

Logistic Regression

- In binary classification with logistic regression, predictions are positive ($\hat{y}^i = 1$) if $P(\hat{y}^i = 1 \mid x_i, w) > 0.5$; otherwise, they are negative ($\hat{y}^i = 0$).
- To train logistic regression, the Maximum Likelihood Estimation (MLE) finds the optimal weight vector (w) by maximizing the likelihood of the training set.
- Weight vectors (w) are updated iteratively using gradient ascent based on the log likelihood, considering adaptive learning rates involving η and prediction confidence.

Logistic Regression

- Limitations and Mitigations
 - Logistic regression limitations:
 - Prone to overfitting if training set is linearly separable.
 - Assumes independence among training tuples.
 - Computational challenges for large datasets.
- Mitigations:
 - Introduce regularization to prevent overfitting.
 - Consider graph-based classification for interrelated data.
 - Use stochastic gradient descent for efficiency with large datasets.

Introduction to Model Evaluation and Selection

- When building a classification model, assessing its accuracy for predicting future, unseen data is crucial.
- Questions arise about what accuracy means, how to estimate it, and which measures are appropriate.
- Methods for evaluating classifier accuracy include holdout and random subsampling, cross-validation, and bootstrap techniques.
- Model selection involves choosing the best classifier among multiple options, and this is discussed in the last two sections.

Metrics for Evaluating Classifier Performance

- Classifier evaluation metrics include accuracy, sensitivity, specificity, precision, F1, and $F\beta$.
- Accuracy measures the percentage of correctly classified test set tuples.
- Sensitivity (recall) measures the proportion of positive tuples correctly identified.

Metrics for Evaluating Classifier Performance

- Specificity measures the proportion of negative tuples correctly identified.
- Precision is a measure of exactness, indicating what percentage of predicted positive tuples are correct.
- F_1 and F_β are harmonic mean-based metrics combining precision and recall.

Handling Class Imbalance

- In cases of class imbalance, where one class is rare, accuracy may not be an appropriate metric.
- Sensitivity and specificity are better suited for assessing the recognition of positive and negative tuples.
- Class imbalance can lead to misleadingly high accuracy.

Additional Classifier Comparison Aspects

- Beyond accuracy-based measures, classifiers can be compared in terms of speed, robustness, scalability, and interpretability.
- Speed considers computational costs.
- Robustness assesses performance with noisy or missing data.
- Scalability evaluates efficiency with large data.
- Interpretability measures the comprehensibility of the classifier.

Evaluation Methods: Holdout and Random Subsampling

- The holdout method randomly splits data into training and test sets (typically $2/3$ and $1/3$).
- Accuracy is estimated using the test set, providing a pessimistic estimate due to limited training data.
- Random subsampling repeats the holdout method k times and averages accuracy estimates for a more robust result.

Evaluation Method: Cross-Validation

- In k-fold cross-validation, data is divided into k subsets; training and testing are performed k times.
- Each subset is used once as a test set while the rest are used for training.
- Accuracy is calculated as the overall number of correct classifications divided by the total number of tuples.

Evaluation Method: Bootstrap

- The bootstrap method involves random sampling of training tuples with replacement.
- Typically, around 63.2% of the original data ends up in the training set, while the rest forms the test set.
- This process is repeated k times, and accuracy is estimated as a weighted average of test and training set results.
- Bootstrapping can be overly optimistic and is best suited for small data sets.

Model Selection Using Statistical Tests of Significance

- To determine the best model among M1 and M2:
 - Calculate mean error rates for both models through 10-fold cross-validation.
 - Note that mean error rates alone might not be sufficient due to variance.
- Statistical significance testing is needed to identify a genuine difference in error rates.

Model Selection Using Statistical Tests of Significance

- Obtain confidence limits for mean error rates.
- Perform 10-fold cross-validation multiple times for each model.
- Error rates from cross-validation form independent samples following a t-distribution with $k - 1$ degrees of freedom ($k = 10$).
- Hypothesis testing using a t-test helps decide if models are significantly different.

Model Selection Using Statistical Tests of Significance

- Calculate the t-statistic based on the difference in mean error rates.
- Consult a t-distribution table for significance levels (e.g., 5% or 1%).
- If $t > z$ or $t < -z$ (where $z = \text{sig}/2$), models are significantly different.
- For two test sets instead of one, use the two-sample t-test, and take the minimum degrees of freedom.

Comparing Classifiers Based on Cost-Benefit and ROC Curves

- Assess costs and benefits in classification using true positives, true negatives, false positives, and false negatives.
- Consider the cost of false negatives compared to false positives.
- Assign different costs to errors based on their impact.
- ROC curves depict the trade-off between true positive rate (TPR) and false positive rate (FPR).
- Calculate TPR (sensitivity) and FPR (1-specificity) from a test set.

Comparing Classifiers Based on Cost-Benefit and ROC Curves

- ROC curve plots TPR against FPR for different parts of the test set.
- Area under the ROC curve (AUC) measures classifier accuracy.
- Higher AUC indicates better accuracy.
- Probabilistic classifiers are needed for ROC curve construction.
- Rank and sort tuples by class probability for ROC curve plotting.
- Assess classifier accuracy by measuring the AUC.

Techniques to Improve Classification Accuracy

- Ensemble methods aim to enhance classification accuracy.
- Ensembles combine multiple classifiers, and their votes decide the final prediction.
- Bagging, boosting, and random forests are common ensemble methods.
- Bagging involves creating multiple training sets through bootstrap sampling.
- Bagged classifiers reduce variance and improve accuracy.

Techniques to Improve Classification Accuracy

- Boosting assigns weights to training tuples and iteratively focuses on misclassified tuples.
- Boosted models tend to achieve high accuracy but can overfit.
- Random forests consist of decision tree classifiers with random attribute selection.
- They are robust, insensitive to attribute selection, and efficient for large datasets.

Improving classification accuracy of class-imbalanced data

- Class imbalance occurs when the positive class is rare in two-class or multiclass data.
- Solutions include oversampling, undersampling, threshold moving, and ensemble methods.
- Oversampling replicates positive instances, while undersampling reduces negative ones to balance data.
- Threshold moving adjusts decision thresholds for classifiers, and ensemble methods combine techniques for addressing class imbalance.

Summary

- Classification involves data analysis to create models predicting categorical labels.
- Decision tree induction is a top-down algorithm using attribute selection measures.
- Naïve Bayesian classification assumes class-conditional independence based on Bayes' theorem.
- Linear classifiers predict labels using a linear combination of attribute values.
- Eager learners like decision tree, Bayesian, and linear classifiers build models before classifying.

Summary

- Confusion matrix evaluates classifiers, measuring true positives, true negatives, false positives, and false negatives.
- Classifier evaluation uses data partitioning methods like holdout, random sampling, cross-validation, and bootstrapping.
- Significance tests and ROC curves aid in model selection.
- Ensemble methods enhance accuracy by combining multiple base classifiers.
- The class imbalance problem arises when the main class is underrepresented, with solutions like oversampling, undersampling, threshold moving, and ensemble techniques.